

SESSION 7

Django Model Forms

Topics Covered: ModelForm benefits, Automatic form generation, Saving form data to model

Activities: Create DoctorForm, Save doctor details from form

Task 1: Create Django model Restaurant with fields name, cuisine, rating, then generate ModelForm named RestaurantForm for it.

Explanation & Approach: *ModelForm automatically binds HTML inputs to model database fields, reducing boilerplates.*

```
# foodiespot/forms.py
from django import forms
from .models import Restaurant

class RestaurantForm(forms.ModelForm):
    class Meta:
        model = Restaurant
        fields = ['name', 'cuisine', 'rating', 'location']
```

Task 2: Use RestaurantForm to build a view that displays form and saves new restaurant entries to database when submitted.

Explanation & Approach: *form.save() automatically instantiates and persists a new model record directly to SQL database tables.*

```
# foodiespot/views.py
from django.shortcuts import render, redirect
from .forms import RestaurantForm

def create_restaurant(request):
    if request.method == 'POST':
        form = RestaurantForm(request.POST)
        if form.is_valid():
            form.save() # Saves object to database
            return redirect('restaurant_list')
    else:
        form = RestaurantForm()
    return render(request, 'foodiespot/restaurant_form.html', {'form': form})
```

Task 3: Customize RestaurantForm to add placeholder for name field and set minimum value for rating to 1 and maximum to 5.

Explanation & Approach: *The Meta class widgets dictionary enables custom HTML attributes (placeholders, min, max, step).*

```
# foodiespot/forms.py (Widget Customization)
from django import forms
```

```

from .models import Restaurant

class RestaurantForm(forms.ModelForm):
    class Meta:
        model = Restaurant
        fields = ['name', 'cuisine', 'rating', 'location']
        widgets = {
            'name': forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'Enter
Restaurant Name (e.g. Olive Garden)'}),
            'rating': forms.NumberInput(attrs={'class': 'form-control', 'min': '1.0',
'max': '5.0', 'step': '0.1'}),
            'location': forms.TextInput(attrs={'class': 'form-control', 'placeholder':
'City / Area Location'}),
        }

```

Task 4: Add validation to RestaurantForm so name must be at least 3 characters and rating between 1 and 5.

Explanation & Approach: *Combining Meta widgets with clean_fieldname methods yields robust user experience and server validation.*

```

# foodiespot/forms.py (Complete Validation)
from django import forms
from django.core.exceptions import ValidationError
from .models import Restaurant

class RestaurantForm(forms.ModelForm):
    class Meta:
        model = Restaurant
        fields = ['name', 'cuisine', 'rating', 'location']

    def clean_name(self):
        name = self.cleaned_data.get('name')
        if len(name) < 3:
            raise ValidationError("Restaurant name must contain at least 3 characters.")
        return name

    def clean_rating(self):
        rating = self.cleaned_data.get('rating')
        if rating < 1.0 or rating > 5.0:
            raise ValidationError("Rating must be strictly between 1.0 and 5.0.")
        return rating

```